

Planejamento e Roteiro - Projeto Final de IA (RAG)

1. Análise do Documento

O documento descreve os requisitos para a avaliação final da disciplina "Tópicos em Inteligência Artificial", focando em Inteligência Artificial Generativa e RAG (Retrieval-Augmented Generation).

Objetivo Principal: Criar um chatbot RAG simples que responda perguntas baseado em uma base de documentos fornecida pelo aluno, citando as fontes.

Requisitos Técnicos:

- **Base Documental:** Mínimo de 5 textos curtos ou 1 documento grande dividido.
- **Processamento:** Textos devem ter conteúdo e metadados (fonte, área, etc).
- **Pipeline RAG:**

1. Divisão em chunks.
2. Geração de embeddings.
3. Armazenamento em Vector Store (FAISS recomendado).
4. Retriever para buscar trechos relevantes.
5. Geração da resposta com LLM usando o contexto.
6. Exibição das fontes na resposta.

Testes Obrigatórios (5 perguntas):

- 3 perguntas com respostas claras nos textos.
- 1 pergunta com palavras diferentes (testar busca semântica).
- 1 pergunta sem resposta na base (o sistema deve informar que não há informação suficiente, evitando alucinações).

Entregáveis:

1. Código fonte (.py, .ipynb ou link do GitHub).
2. Relatório em PDF (2 a 4 páginas) com: tema, resumo do pipeline, configurações (modelo, k, interface), tabela com os testes, comentários finais.
3. Prints da aplicação funcionando.

2. Roteiro de Implementação (Passo a Passo)

Passo 1: Definição do Tema e Coleta de Dados

- [] Escolher um tema (Ex: Estatuto da Universidade, Políticas de RH, Manuais de um software).
- [] Coletar 5 textos curtos ou 1 texto longo sobre o tema.
- [] Estruturar os documentos adicionando metadados (ex: ``{"fonte": "Estatuto_Cap_1", "secao": "Regras de Matricula"}``).

Passo 2: Configuração do Ambiente e Pipeline

- [] Instalar as bibliotecas necessárias: `langchain`, `faiss-cpu`, `openai` (ou outro LLM), `sentence-transformers` (para embeddings locais se preferir), `streamlit` (para interface).
- [] Carregar os documentos (ex: usando `Document` do Langchain).
- [] Configurar o **Text Splitter** para dividir os textos em chunks (ex: `RecursiveCharacterTextSplitter`).

Passo 3: Embeddings e Vector Store (FAISS)

- [] Escolher o modelo de embedding (Ex: `all-MiniLM-L6-v2` do HuggingFace ou `text-embedding-3-small` da OpenAI).
- [] Gerar os embeddings dos chunks.
- [] Salvar os embeddings no FAISS (`FAISS.from\_documents`).

Passo 4: Retriever e Geração de Respostas

- [] Configurar o retriever no FAISS definindo o valor de `k` (ex: `k=3` para retornar os 3 chunks mais relevantes).
- [] Configurar o LLM (OpenAI, Gemini, etc) com um **Prompt Template** rigoroso que instrua o modelo a responder **apenas** com base no contexto, e caso não saiba, avisar que não há informação suficiente.
- [] Montar a cadeia (chain) que recebe a pergunta, busca no FAISS e gera a resposta com as fontes.

Passo 5: Interface (Streamlit)

- [] Criar um arquivo `app.py` usando Streamlit.
- [] Adicionar um campo de input para a pergunta do usuário.
- [] Mostrar a resposta gerada e logo abaixo listar os metadados (fontes) dos chunks utilizados.

Passo 6: Testes Obrigatórios

- [] Elaborar as 5 perguntas conforme os requisitos.
- [] Executar e validar se o sistema acerta as respostas e lida corretamente com a pergunta fora do contexto (informando que não sabe).
- [] Tirar prints da tela com os resultados de cada teste.

Passo 7: Elaboração do Relatório Final (PDF)

- [] Escrever o relatório (2 a 4 páginas).
- [] Incluir Tema, Descrição da base, Resumo do Pipeline, Configurações usadas (Modelo de embedding, K, Ferramenta de interface).
- [] Preencher a tabela com os 5 testes (Pergunta, Resposta Obtida, Fontes).
- [] Escrever o comentário final (o que funcionou, o que pode melhorar).
- [] Exportar para PDF.

3. Planejamento das Próximas Ações

Para começar o desenvolvimento da parte prática do trabalho, siga a seguinte ordem:

1. **Definição da Base:** Qual será o tema dos documentos que você quer utilizar? Por favor, providencie os textos ou o PDF de origem.
2. **Setup do Código:** Criaremos um script Python ou Notebook para validar o carregamento e divisão (chunks) do documento.
3. **LLM:** Defina qual API de LLM utilizaremos (OpenAI, Gemini, Ollama local, etc).
4. **Construção:** Programaremos o RAG e em seguida a interface web simples usando Streamlit.